



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Compilado de Preguntas

Arquitectura de Sistemas de Software (IIC2173)

2024-2

Recopilación por el equipo docente 2024-2

Índice

1.	Verdadero o Falso	3
a.	Patrones y Estilos	3
i.	“Características Pub-Sub” (I1-2022-1).....	3
2.	Desarrollo Escrito	3
a.	NFR	3
i.	“Comisaría Virtual” (I1-2022-1)	3
ii.	Parte de la pregunta “Tracking de fechorías (parte 1)” (I1-2022-1)	4
iii.	Parte de la pregunta “Sistema de drones en una caverna (parte 1)” (I1-2022-1)	4
iv.	“MichiGuardian” (I1-2021-2).....	4
b.	Patrones y Estilos	5
1.	“Tracking de fechorías (parte 1)” (I1-2022-1)	5
2.	“Tracking de fechorías (parte 2)” (I1-2022-1)	6
3.	“Sistema de drones en una caverna (parte 1)” (I1-2022-1).....	6
i.	Parte de la pregunta “MichiGuardian” (I1-2021-2).....	8
b.	Conectores.....	8
i.	Parte de la pregunta “Sistema de drones en una caverna (parte 1)” (I1-2022-1)	8
3.	Diagramas UML.....	9
a.	Modelación y Visualización.....	9
i.	Parte de la pregunta “MichiGuardian” (I1-2021-2).....	9
ii.	Parte de la pregunta “MichiGuardian” (I1-2021-2).....	9
iii.	“Fundación SCP” (I1-2021-2).....	9
4.	Desarrollo completo de pruebas pasadas	9
a.	P2 I1-2023-2: “Sistema de alarmas multifuncionales”	9
b.	P2 I1-2022-2: “The hive and Fnasa”	12

Nota: las preguntas completas llevan el formato <comillas><paréntesis>. Los ítems que tienen el formato *Parte de la pregunta*<comillas><paréntesis> indican que hay un enlace hacia dicha pregunta completa.

1. Verdadero o Falso

a. Patrones y Estilos

i. “Características Pub-Sub” (I1-2022-1)

¿Qué frases son ciertas sobre una implementación pub-sub?

- Los suscriptores siempre responden al publicador del mensaje
 - No necesariamente. Un logger por ejemplo no tiene porque responder a cada mensaje
- No posee limitaciones de escalabilidad al usar un servidor Broker
 - Un solo servidor broker puede no bastar para tantas peticiones. De hecho, a veces necesita apoyo como un Hub o clustering.
- Es posible que en algunas implementaciones se pierdan mensajes
 - Completamente factible. Ver QoS.
- Es necesario que los mensajes sean sumamente detallados y suelen ser pesados
 - Definitivamente no, además que cierta complejidad del manejo lo toma el mismo broker. Ver MQTT. De hecho se privilegian los mensajes pequeños y ligeros, al haber tantos en muchas implementaciones.
- Es altamente flexible a cambios de tipo de mensaje y estructura
 - Usualmente no. Es complejo cambiar la estructura puesto que hay que actualizar todos los clientes y pub-sub no incluye provisiones para esa flexibilidad. Es posible incluso que en algunas implementaciones queden fragmentados los clientes por pérdida de mensajes.

2. Desarrollo Escrito

a. NFR

i. “Comisaría Virtual” (I1-2022-1)

Desde hace unos años existe un servicio web llamado "Comisaría virtual". En él, se pueden realizar múltiples trámites que requieren la identidad del usuario, principalmente con el fin de emitir permisos críticos en contexto de pandemia, denuncias y otros trámites.

¿Qué requisitos no funcionales son críticos para el funcionamiento de esta plataforma?

- Escalabilidad
- Disponibilidad
- Flexibilidad
- Seguridad
- Mantenibilidad
 - No se mantenía demasiado porque tantos cambios no habían. Por esto, es el menos importante de los 5.

- ii. [Parte de la pregunta “Tracking de fechorías \(parte 1\)” \(I1-2022-1\)](#)
- iii. [Parte de la pregunta “Sistema de drones en una caverna \(parte 1\)” \(I1-2022-1\)](#)
- iv. **“MichiGuardian” (I1-2021-2)**

Las pérdidas de mascotas son ocurrencias muy complejas para sus dueños. Según estadísticas, 1 de cada 3 mascotas se perderán al menos una vez a lo largo de su vida. Usted, junto a otros compañeros de trabajo de *LegitBusiness* abandonan sus puestos de hámsters corporativos para transformarse en todos unos emprendedores, fundando MichiGuardian, un collar con Bluetooth y una app a juego para encontrar más rápido a las mascotas perdidas. El collar constantemente está emitiendo una señal con identificación via BLE (una variante de bluetooth que consume menos energía), la cual pueden identificar usuarios con la app instalada. Cuando un gato con el collar marcado como perdido, contacta con uno de estos aparatos con la app instalada, se notifica al dueño la posición aproximada del gato discretamente. En caso de que más usuarios encuentren esa mascota, se unificarán sus datos y se triangulará su posición.

Sus cofundadores le piden que modele una beta de este sistema, considerando que la app tendrá en beta aproximadamente 10.000 usuarios. El sistema debe poder notificar a los usuarios, llevar seguimiento de las mascotas, conectarse con el servicio de notificaciones de Google y Apple, así como incorporar tecnologías serverless para los servicios que usted considere apropiados.

Debe modelar el sistema que va el teléfono y todo el backend en la nube, de una forma agnóstica al proveedor de nube.

- a) Indique hasta 3 atributos de calidad que Ud. considere son fundamentales en este problema. Enumérelos en orden de prioridad. Justifique porque hizo esta elección.

RNFs claves en la solución:

- Escalabilidad: Naturalmente, el requisito de 10000 usuarios invoca el problema de escalar la solución de alguna forma, con balanceo de carga u otros métodos.
- Performance: Debe haber consideraciones sobre el número de usuarios y la velocidad de notificaciones.
- Disponibilidad: Las notificaciones son importantes así que debe poder responderse lo antes posible y debe poder atender las peticiones.
- Confiabilidad: Son mensajes importantes así que los errores deben mantenerse al mínimo.
- Usabilidad: Posee muchas interfaces que el usuario debe poder usar e interactuar bien.
- Integrabilidad: poco, puesto que solo trabaja con su sistema y de forma muy definida.
- Cualquier otro que esté muy muy bien justificado. Está mal derechamente si usan reusabilidad, portabilidad, mantenibilidad, testeabilidad, puesto que son muy menores o no importantes.

- b) Indique los estilos arquitectónicos (al menos 2) que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada patrón (responsabilidad).

- Pub-Sub es necesario para las notificaciones, y código basado en eventos.
- Cli-Srv para las cosas básicas.
- Cualquier otro bien justificado.
- Pizarra no aplica.

c) Haga un diagrama de bloques indicando los componentes arquitectónicos y sus relaciones que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo el desarrollo anterior expresado soluciona el problema planteado.

- Depende de la solución del alumno: puede variar. Se espera el mayor detalle posible y comentarios sobre los componentes, decisiones y conectores

Ejemplo de solución:

https://drive.google.com/file/d/1V-4OnzGRvx7Sme_8O8sK58LUL-mM8LL_/view?usp=sharing

d) ¿Qué atributos de calidad (al menos 1) se ven perjudicados en su solución?

- Depende de la solución del alumno: puede variar. Debe tener sentido y no colapsar con lo propuesto anteriormente.

e) Aún sin un modelo de negocios claro y viviendo de hablar Corfiano (del planeta Corfo...), eventualmente ustedes llaman la atención de Apple y hacen un convenio con ellos (frente a todo pronóstico) para poder ocupar su red de iPhones para hacer las triangulaciones y hallazgos. Apple les ofrece notificar a un endpoint que uds provean para enviar los eventos de hallazgo. Incorpórelo en una variante de su diagrama

- Deben incorporar el endpoint con un adaptador a un sistema de notificaciones pub-sub, con un adaptador funcional que pueda hacer las interacciones, de preferencia un worker asíncrono que se encargue de llevar las peticiones

Ejemplo de solución:

https://drive.google.com/file/d/19TPd2kAJOuXH5ZuAhZV6PgA6mD_ovKmp/view?usp=sharing

b. Patrones y Estilos

1. “Tracking de fechorías (parte 1)” (I1-2022-1)

El modelo de cobro de APIs por uso es uno muy común para APIs que ofrecen servicios de uso puntual y gran valor. Es una práctica común de uso en la industria, y sin ir más lejos, Twitter tiene su propia API premium, la que ofrece diversos niveles de información sobre su plataforma. Pagando cierta cantidad de dólares al mes, es posible obtener un número específico de requests que se pueden hacer para obtener información.

Usted desea publicar una API construida capaz de emitir información sobre las ubicaciones conocidas y noticias pasadas acerca de las fechorías de un ratón blanco conocido como “Stuart Little”. Debido a que el vil roedor es conocido por sus crímenes, mucha gente lo busca, así que espera que su API tenga un alto nivel de consumo. Se espera que debido a la importancia del tópico, sus requests sean bastantes (100-400 peticiones/segundo, más de lo que puede manejar un solo servidor en este caso).

Debe diseñar una solución que sirva de “Gatekeeper” de su API. Además de usar una API gateway, lo más básico, debe incorporar capacidades de autorización en base al uso de la API, log de requests para análisis posterior y un receptor de pagos bitcoin. Asuma que a cada usuario se le entrega un token con un tiempo de vida infinito, que permite autenticarlo

Considere que su solución está en la nube pero tiene poco dinero, así que tiene accesos solamente a servicios básicos cloud (los que están en el enunciado de la E1)

- ¿Qué justifica el uso de API gateway en este caso?
 - La frase Gatekeeper y las autorizaciones. Además, se espera que la API esté bien monitoreada
- Confeccione un diagrama de componentes UML indicando los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado.
 - Depende del alumno.

2. “Tracking de fechorías (parte 2)” (I1-2022-1)

Dado el éxito de la API, la demanda aumentó muchísimo y para cubrir ese crecimiento, ha replicado sus servicios en todo el mundo, en clusters de servidores por región, cada uno con sus datos, su API gateway y contador de usos. No obstante, hay usuarios que abusan de los endpoints públicos y los estresan. Un acercamiento común es denegar las conexiones por IP de origen.

Tiene como tarea modelar un sistema capaz de informar a todos los endpoints cuando un usuario ha sido baneado por abusar de los endpoints públicos, así como tener un sistema de logging unificado para seguir las peticiones y verificar datos de rendimiento

- ¿Que estilo/subestilo/patrones puede usar para mantener operando un sistema de logging unificado, teniendo tantas instancias activas? ¿Qué implicancias tiene su elección?
 - Pubsub y sus implicancias
- Indique (al menos 3) atributos de calidad que Ud. considere son fundamentales en este problema. Enumérellos en orden de prioridad.
 - [más prioritario] --
 - Performance
 - Escalabilidad
 - Mantenibilidad
 - [menos prioritario] --
- Indique los estilos arquitectónicos y sus detalles (al menos 2) que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).
 - Depende del alumno.
- Extienda el diagrama de la sección anterior para incorporar estos cambios
 - Depende del alumno.

3. “Sistema de drones en una caverna (parte 1)” (I1-2022-1)

Como parte de una colaboración con una expedición científica a una red de cavernas, se le encargó crear una sistema de comunicación para drones capaz de posicionarse a lo largo de una caverna, y mediante sensores especiales al aterrizar, registrar eventos de ruido y/o temblores y emitirlos a la estación de control. Como componentes para crear esta red, le facilitaron aproximadamente 100 drones y una estación de control de mediana potencia (no puede ser mucho debido a que es equipo

portátil). Al esparcir los 100 drones todos idénticos con sensores y alcance de 120m de radio, de forma secuencial en su posición final, queda la siguiente distribución en la red de cavernas:

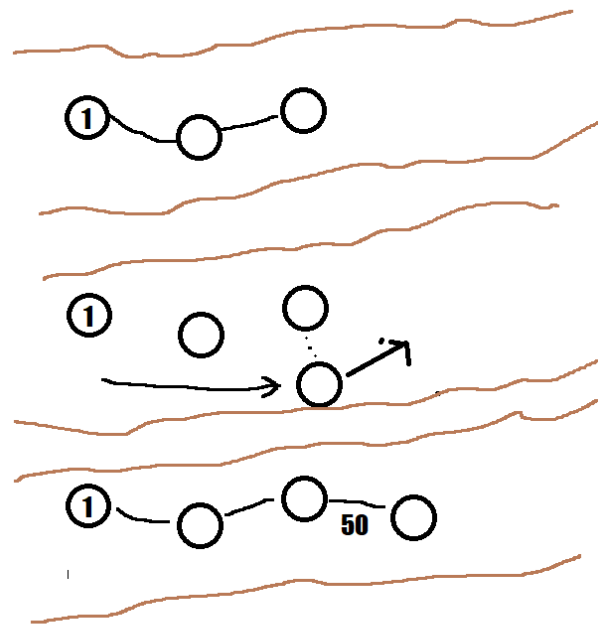


Figura 1: “Representación drones avanzando”

Nótese que los drones no están posicionados en algún orden específico.

Su misión es modelar un sistema tal que sea capaz de conectar los drones cuando uno esté a 50 m de distancia del otro, sean capaces de conectarse de manera encadenada y devolver la información de eventos a la estación de control, así como proponer un motor de decisión para elegir las alertas que se van a enviar. Además, debe asegurarse de que ambos drones puedan comunicarse bien seleccionando un ancho de banda compatible con la distancia.

Asuma que el software de ruteo funciona y se puede usar como un componente (una vez conectados los drones, el paquete sabrá dónde ir si pasa por un ruteador).

- Sus drones poseen una capacidad de CPU limitada, y se conectan mediante señales de radio de baja potencia y baja tasa de datos (bit/s). ¿Cómo se puede afrontar esta falta de capacidad de la red para transmitir datos en masa?
 - Datos en binario, poco formato, compresión...
- Defina el o los conectores necesarios para la comunicación entre drone y drone. Indique todos los parámetros requeridos para definir bien el conector, y justifique detalladamente su elección.
 - Enlace/binding. Establecimiento de callbacks, señalar que los eventos son necesarios y parte del tipo de emisión. Nótese que la naturaleza de “eventos” motiva cierto tipo de conectores.
 - Se requiere un árbitro, fault handling, autoritario, capaz de hacer locks.
- Indique 3 pros y 2 contras de su elección de conector

- Depende del alumno.
- Modele un diagrama de secuencia del proceso de conexión inicial en base a la definición de su sistema. Este proceso se establece cuando un drone se posiciona a aprox 50 metros del drone anterior. Recuerde que inicialmente los drones no se conocen y el drone que llega solamente sabe que es el último de esa rama.
 - La respuesta por alumno puede variar, pero es más o menos así:
El último drone le solicita un callback al drone y se registra con este. Esto permite que el drone sepa a quién emitir llamadas o recibir requests, y que sea on-the-fly. Además, hay una negociación con coordinación para ver el ancho de banda en base a “pings”, probando hasta que la señal no sea óptima. Este sistema recuerda en parte a las conexiones dial-up, donde se hace un handshake, se negocia un ancho de banda y se establece una conexión. No obstante, las limitaciones de ancho de banda y emisiones de eventos no determinísticos hacen necesario el uso de un callback.
- Indique (al menos 3) atributos de calidad que Ud. considere son fundamentales en este problema. Enumérelos en orden de prioridad.
 - [más prioritario] --
 - Escalabilidad: Poder conectar bien decenas de drones.
 - Confiabilidad: Para emitir los mensajes. Muy bien si menciona QoS o algún mecanismo de confiabilidad.
 - Performance: Para manejar los mensajes en un entorno embebido, el sistema debe ser eficiente.
 - Flexibilidad.
 - Cualquier otro bien justificado.
 - [menos prioritario] --
- Indique los estilos arquitectónicos (al menos 2) que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).
 - N-Capas
 - Cualquier otro bien justificado
- Confeccione un diagrama de componentes UML indicando los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado. Modele el sistema de red y el sistema de recolección de datos con el motor de decisión de reporte.
 - La respuesta varía por alumno.

i. [Parte de la pregunta “MichiGuardian” \(I1-2021-2\)](#)

b. Conectores

i. [Parte de la pregunta “Sistema de drones en una caverna \(parte 1\)” \(I1-2022-1\)](#)

3. Diagramas UML

a. Modelación y Visualización

- i. [Parte de la pregunta “MichiGuardian” \(I1-2021-2\)](#)
- ii. [Parte de la pregunta “MichiGuardian” \(I1-2021-2\)](#)
- iii. [“Fundación SCP” \(I1-2021-2\)](#)

Dentro de la enorme lista de anomalías que conserva la Fundación SCP (Secure, contain and protect), está una de las más notables, SCP-079, un ordenador con una IA avanzada y peligrosa que automejoró en una máquina de capacidades muy rudimentarias.

La fundación, en miras a explorar las capacidades de SCP-079 desea ver qué sucede si a la máquina se le concede cierto grado de acceso a internet. Para esto, se le especifica que diseñe un sistema que revise la correctitud de las peticiones y autorice las conexiones a las fuentes de datos, que transforme en inocuos aquellos intercambios de información que puedan ser dañinos, además de notificar estas peticiones a varios investigadores que supervisan este experimento. Como investigador novato, te piden que modelas un diagrama de componentes UML que describa un sistema así y comente su solución.

- Esta respuesta debe contener una solución que describa un sistema con Pipe and Filter. La idea es que el Pipe And Filter implemente varios mecanismos de seguridad que limiten el acceso, impidan excepciones y filtren los inputs/outputs de los requests/responses que pueda tener acceso el ente contenido detrás de su solución. Además, componentes filter cruciales podrían generar notificaciones. Debe haber un sistema de notificaciones asociado y ojalá una abstracción de varios filtros secuenciales (ver diagrama ejemplo).

Ejemplo de solución:

<https://drive.google.com/file/d/1-ZtzYMIOW-p81JDxydQIK75RfMHzaI5p/view?usp=sharing>

¿Es posible implementar serverless en un problema así? Si tu respuesta es positiva, describe cómo podrías implementarlo en tu solución propuesta. En caso contrario, argumenta el porqué no.

- Es en efecto una posibilidad puesto que las demandas podrían aumentar arbitrariamente a gusto de los investigadores, lo cual en un entorno *on premises* sería sumamente difícil de escalar. Las reglas y chequeos podrían ser implementados en forma de funciones atómicas que consultan bases de datos de reglas, con una alta escalabilidad preparada para enfrentar esta alza. No obstante, hay que tener ojo con el posible abuso del agente contenido por la solución, e implementar limitadores de requests y algún monitoreo constante (no necesitan especificar cómo pero sí que se haga).

4. Desarrollo completo de pruebas pasadas

a. P2 I1-2023-2: “Sistema de alarmas multifuncionales”

Durante los últimos años, las personas han mostrado creciente preocupación por la seguridad de sus hogares, haciendo que los servicios de alarmas y monitoreo se conviertan en una necesidad. En este contexto, la empresa LegitBusiness creó una nueva empresa para abordar este problema:

OmniFlyEyes, que se propone como un sistema de grabación y monitoreo aéreo de las casas y hogares que lo contraten. La empresa Esperan que ustedes les presenten un diseño de su arquitectura de software.

La idea es que, mediante drones, se pueda sobrevolar las casas y edificios para realizar grabaciones 24/7 de las zonas especificadas (por ejemplo, la propiedad y su patio) como “avistadores”. Estos drones contarán con un módulo de IA (altamente eficiente en consumo) que expondrá las interfaces iGetVideo e iSendInfo. La primera obtendrá las grabaciones en tiempo real y la segunda enviará los datos analizados también en tiempo real, señalando si se detectan personas o autos cerca y si hay vulneraciones a la propiedad.

Para coordinar los drones “avistadores”, LegitVision contará con centros de control por comuna. Dado que una comuna puede cubrir un área extensa y la cobertura no siempre es constante en muchas ciudades, para asegurar que ningún dron pierda la señal, estos deberán ser capaces de elegir dentro de un grupo a un dron que actúe como “relay”. Estos “relays” enviarán la información de su grupo y, además, tendrán la capacidad de coordinarlos e informar a la central si falta alguno o si se requiere un reemplazo.

De esta manera, los drones “avistadores” enviarán la información al dron “relay” de su grupo, el cuál deberá de comunicarse con otros drones “relays” hasta asegurar una conexión con la central de la comuna. Tengan en cuenta que los drones “avistadores” y “relay” son sistemas idénticos con mismas funcionalidades, pero que cambian de rol entre ellos de forma que si un dron “relay” desaparece otro dron “avistador” puede tomar su puesto.

Al realizar el análisis en los mismos drones y delegar carga de transmisión con este sistema, el sistema de centros de control tendrá una carga menor pero deberá procesar cientos de eventos cada hora. Estos eventos se le presentan a un operador humano, el cual debe marcar si son de interés o no. En caso de que sean de interés, el drone recibirá una notificación y se acercará a una torre celular usando el sistema de control del drone que ya tiene un componente “ControlHandler” con una interfaz “IRequestReturn”, con el fin de subir el video completo al centro de control para posterior análisis

4.a) Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos en clases que Ud. considere son fundamentales en este problema. Enumérellos en orden de prioridad y justifique brevemente.

Válidos:

- Disponibilidad
- Eficiencia (por temas de recursos en un dron)
- Confiabilidad
- Escalabilidad
- Elasticidad (solo si se justifica correctamente)
- Performance
- Otro bien justificado

No válidos:

- Portabilidad (todo código funciona en un solo ambiente)
- Usabilidad

- Soporte
- Integrabilidad

4.b) Indique (mínimo 2, pero pueden ser más) los estilos arquitectónicos vistos en clases que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).

Válidos:

- Peer to Peer
- Pub/Sub
- Basado en eventos
- Pipe & Filter
- Otro bien justificado

No válidos:

- Código móvil
- Main y subrutinas (es bloqueante)
- Pizarra
- Intérprete

4.c) Confeccione un diagrama de componentes UML indicando los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado. Incluya anotaciones explicando funcionamiento y supuestos, así como nombres de interfaces

Diagrama de referencia: [11_P2_2023-2.png](#) (este diagrama es mucho más completo de lo que se espera)

Se descuenta por:

- No usar interfaces (-3 pts)
- No usar un estilo priorizado previamente (-4 pts c/u)
- Falta un requisito funcional (-3 pts c/u)
- Componentes desagregados (-1 pto c/u)
- Componentes muy generales (-2 pts c/u) (por ejemplo tienen una api gigante sin desglosar)

No se revisan respuestas que no cumplan con ser un diagrama de componentes UML

4.d) Identifique 2 falencias de su diagrama y comente como las solucionaría y las implicancias de estas soluciones

Variable por cada alumno dependiendo de su respuesta. No se aceptan respuestas que tengan que ver con:

- El cómo hicieron el diagrama
- Que priorizaron algún requisito no funcional y no se ve en su diagrama
- Que les faltó un requisito funcional

Bonus

(+6p) BONUS Esta mañana usted despertó siguiendo cuidadosamente su rutina matutina habitual. Al momento de salir a trabajar, ha notado las siguientes particularidades.

- *En una calle pequeña, los automóviles siguen su ritmo habitual. No obstante, al pasar cerca de una carretera algunos autos desaparecen*
- *Al hablar con otras personas frente a frente, estos no lo entienden: según ellos dice frases cortadas*
- *Árboles especialmente frondosos se vuelven borrosos a la vista*

Los entes encargados de su simulación están en caos, porque los fallos en las simulaciones son inaceptables desde el punto de vista de sus superiores. Para reportar esta incidencia al cónclave, deben identificar los atributos de calidad de la simulación afectados. Indique qué atributo de calidad se ve más afectado por incidente

Respuesta:

- Escalabilidad/Elasticidad/Confiabilidad
- Confiabilidad/Precisión/Interoperabilidad
- Performance/Escalabilidad/Elasticidad

b. P2 I1-2022-2: “The hive and Fnasa”

Dado el creciente aceleramiento en las investigaciones en el espacio, decides unirti a la organización mundialmente conocida como FNASA (Fake NASA) para la creación de nuevas sondas para explorar planetas lejanos.

Como estas exploraciones son altamente costosas en tiempo y dinero, te piden diseñar una propuesta para presentarsela a un equipo de científicos e ingenieros de un sistema capaz de durar como mínimo 4 años en funcionamiento y que pueda obtener muestras de terreno para enviar sus análisis a la Tierra. Para evaluar las distintas propuestas, deciden mostrar las soluciones mediante diagramas UML.

La idea principal, y para esta etapa inicial, es que la sonda esté conformada por una unidad central que será posicionada en una ubicación estratégica en el nuevo planeta, la cuál deberá iniciar una conexión a la Tierra para enviarle los datos al equipo que verá los datos en su dashboard. Esta unidad central, aparte de la comunicación, será “la colmena” de una serie de mini robots diseñados para salir a explorar en líneas rectas e ir cavando agujeros en el piso con la finalidad de extraer pequeñas muestras del terreno, para que mediante un análisis simple filtren e indiquen cuando encuentren minerales interesantes, y así estos marquen la posición y se queden sacando más muestras de esa parte. Hay que tener en consideración que estos mini robots deben saber en todo momento dónde está la unidad central, y que deberían tener la habilidad de mapear el terreno para conocer cuál sería su camino de vuelta. Los mini robots tendrán sensores de sus niveles de batería, para que cuando esta baje, se devuelvan a la unidad colmena para dejar sus datos y que sean enviados a la Tierra.

Finalmente, el equipo encargado de recibir esos análisis deberán poder verlos en un dashboard, y poder darle órdenes a la colmena para que recolecten más información de un punto de interés o para que continúen con el trabajo normal.

- Como el envío de datos es tan costoso entre planetas, indique al menos 1 medida, a nivel de datos o de servicios que puedan estar involucrados, que se debería considerar para perdurar las comunicaciones

R:

- Almacenar los datos y enviarlos juntos como un buffer.
- Comprimir los datos o enviarlos en cadenas de bits más simples para disminuir la cantidad a enviar.
- Usar intermediarios para ir pasando la información (por ejemplo satélites)

- Defina el o los tipos de conectores necesarios para poder manejar la conexión entre la unidad colmena y el dashboard en la tierra. Indique 2 pro y 1 contra de cada uno que defina.

R:

- Stream, los demás dependen de la justificación
- No aplica: Acceso a datos

- Indique (al menos 3) atributos de calidad que Ud. considere son fundamentales en este problema. Enumérelos en orden de prioridad y justifique brevemente.

R:

- Accuracy (debe hacer lo esperado)
- Performance
- Disponibilidad
- Confiabilidad
- Otro bien justificado
- Mantenibilidad (Por que hay costos grandes de enviar equipos desde el planeta y debe durar 4 años)
- **No aplica:**
- Eficiencia (no pedido)
- Integridad
- Seguridad: No se mencionan amenazas de cualquier tipo al sistema y está en prioridad muy reducida respecto a los demás. Usarlo “inventa” la necesidad en el contexto y quita espacio a otros RNF
- Usabilidad

- Indique los estilos arquitectónicos (al menos 2) que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).

R:

- Cliente/Servidor (conexión colmena dashboard)
- Pipe & Filter (Filtro del análisis de minerales)
- Pub/Sub (envío de señales de la colmena a la Tierra o vice versa)
- Basada en eventos
- Otro bien justificado
- **No aplica:**
 - Peer to Peer (no se pide conexión entre ellos)
 - Intérprete
 - Pizarra
- Confeccione un diagrama de componentes UML indicando los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado. Modele el sistema de red y el sistema de recolección de datos con el motor de decisión de reporte.

R:

[Diagrama](#)

La FNASA aprobó su diseño, y se decidió llevarlo a la siguiente fase. En esta parte deberá agregar en su diseño la capacidad de que, si un mini robot no vuelva por algún motivo, la unidad colmena deberá detectar esto y enviar una unidad de reparación a su última posición para intentar salvarlo (recarga de batería, reparaciones de armadura pequeñas, asistencia para volver, etc) o recuperar sus datos y llevarlos a la colmena para saber qué ocurrió.

- Modele el diagrama de secuencia que describa el intento de rescate, desde el momento en que falla el mini robot hasta que es salvado o hasta que se recuperan sus datos. Además, actualice su diagrama de componentes UML con la nueva fase del proyecto.

R:

- Agregar entidades correctamente
- Hacer el flujo de recolección de datos si falla la reparación
- Hacer el flujo de reparación como otro caso
- Agregar el componente y sus conexiones correctamente
- Agregar subcomponente de recolección de datos y conectarlo correctamente
- Agregar subcomponente de reparación y conectarlo correctamente